

Prompts , casos de prueba y conclusión

Estoy implementando en Python un árbol binario de expresión a partir de una expresión en notación postfija. El algoritmo que comprendí es el siguiente:

¡La lógica principal que describiste es **completamente correcta**! Entendiste muy bien la dinámica de cómo una pila permite ir construyendo subárboles y combinándolos a partir de una notación postfija. PDF

Sin embargo, para que tu algoritmo sea robusto en código real, debes considerar algunos casos de error y validaciones esenciales:

Casos y validaciones a considerar

1. Pila vacía / Operandos insuficientes (Bajo flujo):

- **Problema:** Si al leer un operador intentas desapilar (`pop`) el hijo derecho o el izquierdo, pero la pila tiene menos de 2 elementos, la expresión postfija está mal formada (ej. `3 +`). PDF

+ Pregunta a Gemini

Flash

Gemini es una IA y puede cometer errores.

modo (ej. `3 + 3 +`), la expresión contenía operandos de más que faltaban operadores. PDF

- **Solución:** Revisa al final que `pila.tamano() == 1`. Si hay más, la expresión es inválida. PDF + 1

3. Expresión o entrada vacía:

- **Problema:** Si la cadena de entrada está vacía o solo contiene espacios. PDF
- **Solución:** Retorna un mensaje claro o `None` antes de intentar operar. PDF

4. Separación de tokens:

- **Detalle:** Si usas números de varios dígitos (ej. `10 2 +`) o decimales, no puedes recorrer la cadena carácter por carácter. Debes dividir la entrada por espacios (usando `.split()`) para procesar tokens completos. PDF

5. Identificación de operadores:

- Define explícitamente cuáles serán tus operadores soportados (por ejemplo, `+`, `-`, `*`, `/`) para saber con certeza cuándo un token es operador y cuándo es operando. PDF

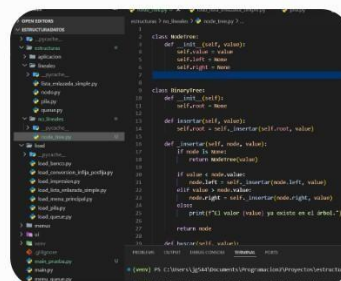
Tu comprensión teórica está al 100%. Cuando estés listo para la implementación del método, me avisas y lo revisamos paso a paso. PDF + 1

👍 🗨️ 🔄 📄 ...

+ Pregunta a Gemini

Flash

Gemini es una IA y puede cometer errores.



Perfecto, acabo de ver tu proyecto.

Veo que ya tienes creada la carpeta `no_lineales` y además ya tienes un árbol binario de búsqueda (`BinaryTree`), por lo que **NO vamos a modificar esa clase**, porque este árbol es diferente al que pide la práctica.

Tu estructura debería quedar algo así:

```
estructuras
├── lineales
│   └── pila.py + Ya existe (la usaremos)
```

+ Preguntar lo que quieras

1. Crea `node_expression.py`

Dentro de `estructuras/no_lineales/`

Python

```
class NodeExpression:

    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Es prácticamente igual al `NodeTree`, pero es mejor tener otra clase porque este árbol no sirve para insertar por comparación de valores.

2. NO uses `BinaryTree`

El archivo que tienes abierto (`node_tree.py`) es un **Árbol Binario de Búsqueda**.

Fíjate que hace esto:

Python

```
if value < node.value:
```

Herramientas

Captura de pantalla
Guardado automático
capturas de pantalla

1. Crea `node_expression.py`

Dentro de `estructuras/no_lineales/`

```
Python
class NodeExpression:

    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Es prácticamente igual al `NodeTree`, pero es mejor tener otra clase porque este árbol no sirve para insertar por comparación de valores.

2. NO uses `BinaryTree`

El archivo que tienes abierto (`node_tree.py`) es un **Árbol Binario de Búsqueda**.

Fíjate que hace esto:

```
Python
if value < node.value:
```

+ Preguntar lo que quieras

```
Python
```

y

```
Python
elif value > node.value:
```

Eso significa que acomoda los nodos **comparando números**.

En el árbol de expresiones **eso nunca ocurre**.

Por ejemplo, para

```
3 4 + 5 2 - *
```

el árbol queda

```
      *
     / \
    +   -
   / \ / \
  3 4 5 2
```

No importa si el 3 es menor que el 5.

El orden depende únicamente de la expresión.

+ Preguntar lo que quieras

```
return None
```

```
return self.top.data
```

```
# Muestra el contenido de la pila
```

```
def print_stack(self):
```

```
    aux = self.top
```

```
    resultado = "TOPE\n!\n"
```

```
while aux is not None:
```

```
    resultado += str(aux.data) + "\n"
```

```
    aux = aux.next
```

```
return resultado
```

 Oferta grati

Perfecto. 🙌 Tu clase `Stack` sí cumple con lo que pide la práctica, porque tiene exactamente los métodos que necesitamos:

-  `push(data)`
-  `pop()`
-  `is_empty()`

No hay que modificar `pila.py`.

aps  Translate  News

Estoy implementando en Python un árbol binario de expresión a partir de una expresión en notación postfija.

El algoritmo que comprendí es el siguiente:

1. Recorrer cada token.
2. Si es un operando, crear un nodo y colocarlo en una pila.
3. Si es un operador, extraer primero el hijo derecho y después el hijo izquierdo.
4. Crear un nodo operador, conectar sus hijos y colocarlo nuevamente en la pila.
5. Al terminar, el único nodo de la pila será la raíz.

Revisa únicamente si la lógica es correcta. No generes todavía el código. Indica posibles errores o casos que debo considerar.

 Ofe

La lógica que describiste es **correcta** y corresponde al algoritmo estándar para construir un árbol binario de expresión a partir de una expresión en notación postfija. El orden de las operaciones permite ir formando subárboles hasta obtener un único árbol cuya raíz representa toda la expresión.  Practica_Arbol_Binario_Expresio...

Algunos casos y detalles que conviene considerar son:

- **El orden de extracción es importante.** Al encontrar un operador, primero se extrae el hijo derecho y después el hijo izquierdo. Si se invierte este orden, la expresión resultante será incorrecta, especialmente en operadores como la resta (`-`) y la división (`/`).  Practica_Arbol_Binario_Expresio...

Algunos casos y detalles que conviene considerar son:

- **El orden de extracción es importante.** Al encontrar un operador, primero se extrae el hijo derecho y después el hijo izquierdo. Si se invierte este orden, la expresión resultante será incorrecta, especialmente en operadores como la resta (-) y la división (/). [Practica_Arbol_Binario_Expresio...](#)
- **Debe haber al menos dos nodos en la pila** cuando se procese un operador. Si no es así, la expresión es inválida porque no existen suficientes operandos para formar un subárbol. [Practica_Arbol_Binario_Expresio...](#)
- **Al finalizar el recorrido debe quedar un solo nodo en la pila.** Ese nodo será la raíz del árbol. Si quedan dos o más nodos, significa que la expresión está incompleta o contiene operandos adicionales. [Practica_Arbol_Binario_Expresio...](#) [Practica_Arbol_Binario_Expresio...](#)
- **Si la pila queda vacía al final,** no fue posible construir un árbol válido. [Practica_Arbol_Binario_Expresio...](#)
- **Es recomendable validar los tokens** para aceptar únicamente operandos y los operadores permitidos (+ , - , * y /). Si aparece un símbolo no reconocido, la expresión debe considerarse inválida. [Practica_Arbol_Binario_Expresio...](#)
- **La expresión debe estar en notación postfija y con los tokens separados por espacios,** ya que el algoritmo procesa un elemento a la vez. [Practica_Arbol_Binario_Expresio...](#)

En conclusión, la lógica del algoritmo es correcta. Lo más importante es respetar el orden de extracción de los hijos, validar el estado de la pila durante el proceso y comprobar que al finalizar exista un único nodo, que será la raíz del árbol de expresión.

     Fuentes

ChatGPT puede cometer errores. Se debe comprobar la información importante

+ Preguntar lo que quieras

Herramienta Re

Captura de pantalla
Guardado automático
capturas de pantalla

Mar

Con base en el algoritmo validado, ayúdame a implementar únicamente el método `build_expression_tree`. Debo utilizar la clase `Stack` que ya existe en mi proyecto. No utilices listas de Python como sustituto de la pila. La clase del nodo se llama `NodeExpression` y tiene los atributos `value`, `left` y `right`. Explica cada parte del método después del código.

Como el profesor pide únicamente el método `build_expression_tree` y que se utilice la clase `Stack` ya existente (sin sustituirla por una lista), este método cumple con esos requisitos. [Practica_Arbol_Binario_Expresio...](#)

Método `build_expression_tree`

Python

```
def build_expression_tree(self, expression):  
  
    stack = Stack()  
  
    tokens = expression.split()
```



+ Preguntar lo que quieras



1. Crear una pila

```
Python  
stack = Stack()
```

Ejecutar

Se crea una pila utilizando la clase `Stack` del proyecto. En ella se irán guardando los nodos y los subárboles que se formen durante el recorrido.

2. Separar la expresión en tokens

```
Python  
tokens = expression.split()
```

Ejecutar

La expresión postfixa se divide por espacios para obtener cada operando y operador por separado.

Ejemplo:

3 4 + 5 2 - *

se convierte en

3
4
+ Preguntar lo que quieras
2

3. Recorrer cada token

```
Python  
for token in tokens:
```

Ejecutar

Se procesa la expresión de izquierda a derecha, analizando un elemento a la vez.

4. Si el token es un operando

```
Python  
if token.isdigit():  
  
    node = NodeExpression(token)  
    stack.push(node)
```

Ejecutar

Cuando el token es un número:

- Se crea un nodo.
- Ese nodo se coloca en la pila.

Todavía no se conecta con ningún otro nodo.

+ Preguntar lo que quieras

5. Si el token es un operador

Python

```
elif token in ['+', '-', '*', '/']:
```

Cuando aparece un operador, significa que ya existen dos operandos o subárboles listos para unirse.

Primero se verifica que la pila tenga suficientes elementos.

Python

```
if stack.is_empty():  
    print("Expresión inválida")  
    return
```

Si la pila está vacía, la expresión no es válida.

6. Extraer los hijos

Python

```
right = stack.pop()  
left = stack.pop()
```

Primero se extrae el hijo derecho y después el hijo izquierdo.

+ Preguntar lo que quieras

Practica_Arbol_Binario_Expresio...

7. Crear el nodo operador

Python

```
operator = NodeExpression(token)
```

Se crea un nodo cuyo valor será el operador (+, -, * o /).

8. Conectar los hijos

Python

```
operator.left = left  
operator.right = right
```

Se asignan los dos nodos extraídos como hijos del operador.

Así se forma un nuevo subárbol.

9. Guardar nuevamente el subárbol

+ Preguntar lo que quieras

10. Obtener la raíz

Al finalizar el recorrido:

Python

```
self.root = stack.pop()
```



Ejecutar

El único elemento que debe quedar en la pila es la raíz del árbol completo.

11. Validar la expresión

Python

```
if not stack.is_empty():  
    print("Expresión inválida")  
    self.root = None
```



Ejecutar

Si después de obtener la raíz todavía quedan elementos en la pila, significa que la expresión está incompleta o contiene operandos de más, por lo que se considera inválida. [Practica_Arbol_Binario_Expresio...](#)



Fuentes

+ Preguntar lo que quieras



Oferta gr

Revisa el siguiente método que construye un árbol de expresión. No lo reemplaces completamente. Identifica los errores, señala las líneas que debo modificar y explica por qué.

```
[from estructuras.lineales.pila import Stack  
from estructuras.no_lineales.node_expression import NodeExpression
```

```
class ExpressionTree:
```

```
    def __init__(self):  
        self.root = None
```

```
    def build_expression_tree(self, expression):
```

```
        stack = Stack()
```

```
        tokens = expression.split()
```

```
        for token in tokens:
```

```
            if token.isdigit():
```



```
                node = NodeExpression(token)
```

+ Preguntar lo que quieras



Después de revisar tu código, la lógica principal para construir el árbol es correcta y sigue el algoritmo solicitado en la práctica. No es necesario reemplazar el método completo; únicamente haría algunas mejoras y validaciones.

1. Validar que la expresión no esté vacía

Ubicación: Al inicio del método `build_expression_tree`.

Agregar:

Python



Ejecutar

```
if not expression.strip():  
    print("La expresión está vacía")  
    return
```

¿Por qué?

Si el usuario solo presiona Enter o escribe espacios, el método intentará trabajar con una expresión vacía.

2. Validar los operandos

Línea actual:



Captur
Guard
captur

2. Validar los operandos

Línea actual:

Python

```
if token.isdigit():
```



Ejecutar

Observación:

Esta línea funciona correctamente para números enteros positivos, por ejemplo:

```
3
15
200
```



Pero no reconocerá:

```
-5
3.5
```



¿Por qué?

El método `isdigit()` solo acepta dígitos.

En esta práctica no es un error, porque el profesor indica que los operandos pueden ser enteros y que admitir decimales es una mejora opcional.

Practica_Arrit_Binario_Expresio...

+ Preguntar lo que quieras



3. Validación antes de hacer `pop()`

3. Validación antes de hacer `pop()`

Actualmente tienes:

Python

```
if stack.is_empty():
    print("Expresión inválida")
    return

right = stack.pop()

if stack.is_empty():
    print("Expresión inválida")
    return

left = stack.pop()
```



Ejecutar

Esta parte es correcta.

Primero verificas que exista el hijo derecho y después verificas que exista el hijo izquierdo antes de desapilarlo. Esto evita errores cuando hay menos de dos nodos en la pila.

4. Orden de los hijos



Her

Captur
Guarda
captur

Python



Ejecutar

```
operator.left = left
operator.right = right
```

No modificar.

Este orden es el correcto y es uno de los puntos más importantes del algoritmo, ya que el primer nodo extraído corresponde al hijo derecho y el segundo al hijo izquierdo.

Practica_Arbol_Binario_Expresio...

5. Validación final de la pila

Actualmente:

Python



Ejecutar

```
self.root = stack.pop()

if not stack.is_empty():
    print("Expresión inválida")
    self.root = None
```

Esta validación es correcta.

Permite detectar cuando sobran nodos en la pila, lo ↓ indica que la expresión está incompleta o tiene operandos adicionales.

Practica_Arbol_Binario_Expresio...

+ Preguntar lo que quieras



Cap
Gua
capt

6. Recorridos (inorder, preorder y postorder)

Permite detectar cuando sobran nodos en la pila, lo que indica que la expresión está incompleta o tiene operandos adicionales.  Practica_Arbol_Binario_Expresio...

6. Recorridos (`inorder` , `preorder` y `postorder`)

Los tres métodos están correctamente implementados.

- `inorder()` genera la expresión infija con paréntesis.
- `preorder()` produce la notación prefija.
- `postorder()` reconstruye la notación postfija.

Cumplen con los recorridos solicitados en la práctica.  Practica_Arbol_Binario_Expresio...

Conclusión

El método `build_expression_tree()` **no necesita ser reemplazado**, ya que la lógica es correcta y cumple con el algoritmo solicitado. La única mejora recomendable es validar que la expresión no esté vacía antes de comenzar el procesamiento. Además, el uso de `token.isdigit()` es suficiente para esta práctica, ya que los casos de prueba utilizan operandos enteros. En general, el código construye correctamente el árbol utilizando la clase `Stack` , respeta el orden de los hijos y realiza las validaciones básicas para detectar expresiones inválidas.

       Fuentes

+ Preguntar lo que quieras



Herran
Captura d
Guardado
capturas c

Permite detectar cuando sobran nodos en la pila, lo que indica que la expresión está incompleta o tiene operandos adicionales.  Practica_Arbol_Binario_Expresio...

6. Recorridos (`inorder` , `preorder` y `postorder`)

Los tres métodos están correctamente implementados.

- `inorder()` genera la expresión infija con paréntesis.
- `preorder()` produce la notación prefija.
- `postorder()` reconstruye la notación postfija.

Cumplen con los recorridos solicitados en la práctica.  Practica_Arbol_Binario_Expresio...

Conclusión

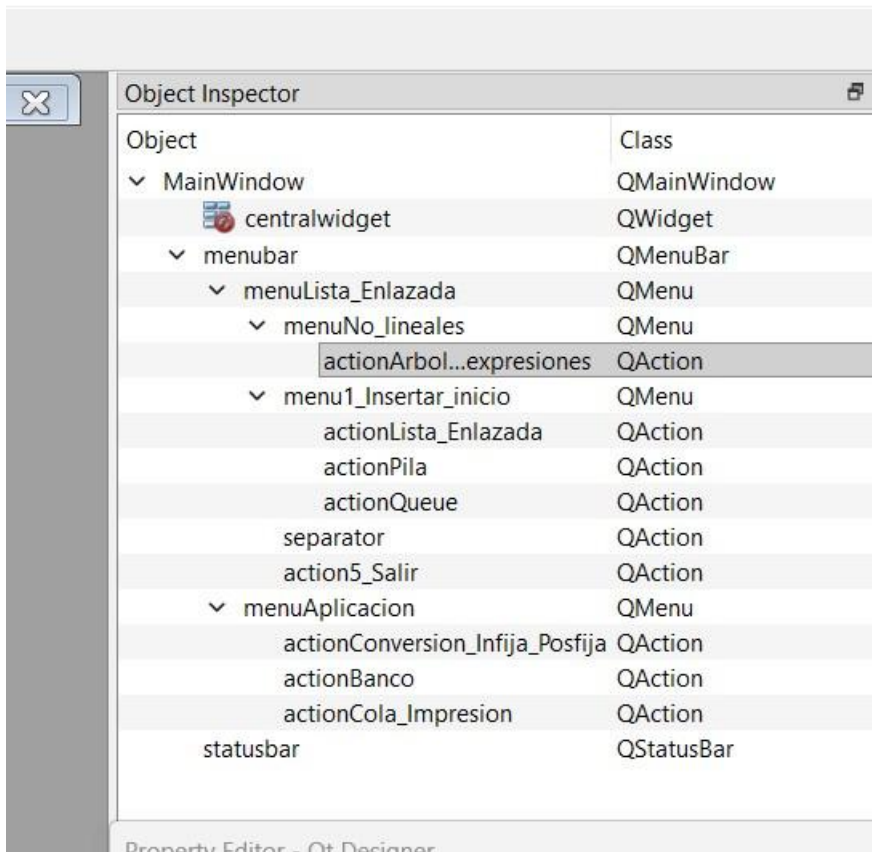
El método `build_expression_tree()` **no necesita ser reemplazado**, ya que la lógica es correcta y cumple con el algoritmo solicitado. La única mejora recomendable es validar que la expresión no esté vacía antes de comenzar el procesamiento. Además, el uso de `token.isdigit()` es suficiente para esta práctica, ya que los casos de prueba utilizan operandos enteros. En general, el código construye correctamente el árbol utilizando la clase `Stack`, respeta el orden de los hijos y realiza las validaciones básicas para detectar expresiones inválidas.

     ...  Fuentes

+ Preguntar lo que quieras



 Herramientas
Captura de pantalla
Guardado de capturas de pantalla



Ahora si lo necesito en QtDesigner

Sí, se puede hacer en **Qt Designer** y, de hecho, quedaría muy bien integrado con el proyecto de **Estructuras de Datos** que has venido desarrollando.

Como ya tienes una estructura con carpetas `ui`, `load`, `estructuras`, etc., haría la aplicación siguiendo ese mismo estilo.

Estructura

```
estructuras/
|
|-- no_lineales/
|   |-- node_expression.py
|   |-- expression_tree.py
|
|-- load/
|   |-- load_expression_tree.py
```

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..



Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

+ Preguntar lo que quieras

Te recomiendo agregar estos componentes:

La interfaz (`expression_tree.ui`)

Te recomiendo agregar estos componentes:

Objeto	Nombre
QLabel	lblTitulo
QLabel	lblExpresion
QLineEdit	txtExpresion
QPushButton	btnConstruir
QLabel	lblInorden
QPlainTextEdit	txtInorden
QLabel	lblPreorden
QPlainTextEdit	txtPreorden
QLabel	lblPostorden

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

Herram

Captura de
Guardado
capturas de

Árbol Binario de Expresiones

Expresión Postfija

+-----+
| 3 4 + 5 2 - * |
+-----+

[Construir Árbol] [Limpiar] [Cerrar]

Inorden

+-----+
| ((3 + 4) * (5 - 2)) |
+-----+

↓

Preorden

+-----+
| * + 3 4 - 5 2 |
+-----+

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

Herram
Captura de
Guardado

Funcionamiento

Cuando el usuario escriba

3 4 + 5 2 - *

📄

y presione

Construir Árbol

📄

el programa hará

↓

Python

`arbol.build_expression_tree(expresion)`

📄

y mostrará

Inorden

`((3 + 4) * (5 - 2))`

📄

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

Herramienta Recortes
Captura de pantalla co
Guardado automáticar
capturas de pantalla.
Marcado

Postorden

+ Preguntar lo que quieras

Mi recomendación

Como ya llevamos varias prácticas con **Qt Designer**, podemos hacer esta aplicación con el mismo nivel de calidad que las anteriores:

- ☒ Menú Principal.
- ☒ Acción **Árbol de Expresiones**.
- ☒ Ventana propia (`QDialog`).
- ☒ Integrada con tu proyecto.
- ☒ Estilo profesional (igual que el de Cola, Pila y Lista).
- ☒ Validación de expresiones inválidas.
- ☒ Botón Limpiar.
- ☒ Botón Cerrar.
- ☒ Mensajes sin `print()` , todo mostrado en la interfaz.

Creo que esta opción se verá mucho más profesional para entregar.



Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

+ Preguntar lo que quieras

Herramienta Recortes

Captura de pantalla copiada
Guardado automáticamente
capturas de pantalla.

Marcado y usado

Sí. Te recomiendo este diseño porque es limpio, profesional y deja espacio para el árbol gráfico.

Configuración del QDialog

objectName: DialogExpressionTree

windowTitle:

Árbol Binario de Expresiones

Size:

1100 x 720

Widgets

Título

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

+ Preguntar lo que quieras

ARQUEO DIARIO DE EXPRESIONES

```
alignment:  
AlignCenter
```

```
font:  
20  
Bold
```

Expresión

QLabel



```
objectName:  
lblExpresion  
  
text:  
Expresión Postfija
```

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..

✦ Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un

[Nuevo chat](#)[Reclamar oferta gratis](#)

Herramienta Recortes

Captura de pantalla copiada en el portapapeles

Caja de texto

Copiar

QLineEdit



```
objectName:  
txtExpresion  
  
placeholderText:  
Ejemplo: 3 4 + 5 2 - *  
  
minimumHeight:  
35
```



Botones

Construir

QPushButton



Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..

✦ Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

[Nuevo chat](#)[Reclamar oferta gratis](#)

Herramienta Recortes

Captura de pantalla copiada en el portapapeles
Guardado automáticamente en la carpeta capturas de pantalla.

[Marcado y uso compartido](#)

+ Preguntar lo que quieras

Botones

Construir

QPushButton



objectName:
btnConstruir

text:
Construir Árbol



minimumSize:
180 x 40

Limpiar

Chats con archivos adjuntos en pausa



Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

Herramienta

Captura de pantalla
Guardado automático
Capturas de pantalla

+ Preguntar lo que quieras

Cerrar

QPushButton



objectName:
btnCerrar

text:
Cerrar



minimumSize:
120 x 40

Línea separadora

Agrega un

Chats con archivos adjuntos en pausa



Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

Herramienta

Captura de pantalla
Guardado automático
Capturas de pantalla

Línea separadora

Agrega un

Line

orientation:
Horizontal



Layout principal

Después de la línea agrega un



QSplitter

Orientation:
Horizontal



Chats con archivos adjuntos en pausa



Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis



Cap
Gua
cap



Preguntar lo que quieras

minimumHeight:
90



QLabel

Recorrido Postorden



QPlainTextEdit



objectName:
txtPostorden

readOnly:
True

minimumHeight:
90



Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..

Recorrido Inorden	Representación gráfica del árbol
((3+4)*(5-2))	
* + 3 4 - 5 2	
3 4 + 5 2 - *	

Mi recomendación

En vez de crear el `.ui` completamente a mano, puedo generarte el archivo `expression_tree.ui` listo para abrir en `Qt Designer`. Solo lo copias a tu carpeta `ui/` y aparecerán todos los widgets, layouts y nombres (`objectName`) ya configurados. Esto te ahorrará bastante tiempo y evitará errores al acomodar los controles.

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..

Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad

Nuevo chat

Reclamar oferta gratis

Sí puedo, pero no en un solo mensaje. Voy a empezar con una versión **compacta** (≈100 líneas) que puedes guardar como `expression_tree.ui` y luego la iremos ampliando.

<> XML

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>DialogExpressionTree</class>
  <widget class="QDialog" name="DialogExpressionTree">
    <property name="geometry">
      <rect>
        <x>0</x>
        <y>0</y>
        <width>1100</width>
        <height>720</height>
      </rect>
    </property>

    <property name="windowTitle">
      <string>Árbol Binario de Expresiones</string>
    </property>
  </widget>
</ui>
```

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..

Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

<string>ÁRBOL BINARIO DE EXPRESIONES</string>

+ Preguntar lo que quieras

XML

```
<widget class="QPushButton" name="btnConstruir">
  <property name="text">
    <string>Construir Árbol</string>
  </property>
</widget>
</item>

<item>
  <widget class="QPushButton" name="btnLimpiar">
    <property name="text">
      <string>Limpiar</string>
    </property>
  </widget>
</item>

<item>
  <widget class="QPushButton" name="btnCerrar">
    <property name="text">
      <string>Cerrar</string>
    </property>
  </widget>
</item>
```

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

XML

```
<widget class="QPlainTextEdit" name="txtInorden"/>
</item>

<item>
  <widget class="QLabel" name="lblPreorden">
    <property name="text">
      <string>Preorden</string>
    </property>
  </widget>
</item>

<item>
  <widget class="QPlainTextEdit" name="txtPreorden"/>
</item>

<item>
  <widget class="QLabel" name="lblPostorden">
    <property name="text">
      <string>Postorden</string>
    </property>
  </widget>
</item>
```

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

Herramienta R

Captura de pant
Guardado auton

<> XML

</ui>

Mi recomendación

Ese XML es una **versión mínima**. Funciona, pero no tiene tamaños, fuentes, espaciados ni proporciones.

Yo puedo generarte un **.ui profesional de unas 500 líneas**, igual que los que hace Qt Designer, con:

- 🎨 Diseño moderno
- 📐 Layouts perfectos
- 🖥️ QGraphicsView grande
- 📁 Panel izquierdo con los recorridos
- 🔵 Botones profesionales
- 🪟 Ventana adaptable
- ✨ Todo listo para conectarlo con `load_expression_tree.py`

Vale mucho más la pena hacerlo así y quedará como si hubiera salido directamente de Qt Designer.

📄 👍 🗨️ ⬆️ ↺️ ⋮

Chats con archivos adjuntos en pausa

Perfecto. Empezaremos con `arbol_expresion.py`. Este archivo contiene la lógica del árbol y los recorridos. Después haremos el `load`.

`estructuras/no_lineales/arbol_expresion.py`

<> Python

```
from estructuras.lineales.pila import Stack
from estructuras.no_lineales.node_expression import NodeExpression

class ArbolExpresion:

    def __init__(self):
        self.root = None

    def build_expression_tree(self, expression):

        self.root = None

        stack = Stack()

        tokens = expression.split()
```

Chats con archivos adjuntos en pausa



Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..
Mejora tu plan para seguir trabajando aquí con archivos e imágenes o inicia un chat nuevo para continuar con un modelo de menor calidad.

Nuevo chat

Reclamar oferta gratis

```
if token.isdigit():
```

+ Preguntar lo que quieras

Python

```
operator.left = left
operator.right = right

stack.push(operator)

else:
    return False

if stack.is_empty():
    return False

self.root = stack.pop()

if not stack.is_empty():
    self.root = None
    return False

return True

def inorder(self):
    return self._inorder(self.root)
```

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..

Herramienta Recortes

```
return ""

resultado = str(node.value)

izquierdo = self._preorder(node.left)
derecho = self._preorder(node.right)

if izquierdo != "":
    resultado += " " + izquierdo

if derecho != "":
    resultado += " " + derecho

return resultado

def postorder(self):
    return self._postorder(self.root)

def _postorder(self, node):

    if node is None:
```

Este archivo ya hace:

- ✓ Construir el árbol desde una expresión postfija.
- ✓ Validar expresiones inválidas.
- ✓ Obtener el recorrido **Inorden**.
- ✓ Obtener el recorrido **Preorden**.
- ✓ Obtener el recorrido **Postorden**.
- ✓ Devolver la raíz mediante `get_root()`, que usaremos para dibujar el árbol en el `QGraphicsView`.

En el siguiente archivo (`load_arbol_expresion.py`) conectaremos la interfaz (`arbol_expresion.ui`) con esta clase, implementaremos los botones **Construir**, **Limpiar** y **Cerrar**, y dibujaremos el árbol gráficamente en el `QGraphicsView`.

diseñaste y con la clase `ArbolExpresion` que hicimos.

Todavía **NO** dibuja el árbol en el `QGraphicsView`. Primero dejaremos funcionando toda la interfaz y después agregaremos el dibujo (es otro bloque de código).

`load/load_arbol_expresion.py`

Python

```
from PyQt5.QtWidgets import QDialog, QMessageBox
from PyQt5.uic import loadUi

from estructuras.no_lineales.arbol_expresion import ArbolExpresion

class LoadArbolExpresion(QDialog):

    def __init__(self):
        super().__init__()

        loadUi("ui/arbol_expresion.ui", self)
```

Perfecto. Ahora vamos a hacer que el árbol se dibuje en el `QGraphicsView`.

Primero agrega estos imports al inicio de `load_arbol_expresion.py`:

Python

```
from PyQt5.QtWidgets import QDialog, QMessageBox, QGraphicsScene
from PyQt5.QtGui import QBrush, QPen
from PyQt5.QtCore import Qt
```

En el `__init__()` agrega

Python

```
self.scene = QGraphicsScene()
self.graphicsView.setScene(self.scene)
```

Quedaría así:

Chats con archivos adjuntos en pausa

Ya usaste todos tus mensajes con Instant hasta 12:38 p.m..

Python

```
self.arbol = ArbolExpresion()

self.scene = QGraphicsScene()
self.graphicsView.setScene(self.scene)

self.btnConstruir.clicked.connect(self.construir_arbol)
self.btnLimpiar.clicked.connect(self.limpiar)
self.btnCerrar.clicked.connect(self.close)
```



En `construir_arbol()`

Después de

Python

```
self.txtPostorden.setPlainText(self.arbol.postorder())
```

Ahora agrega estos dos métodos debajo de `limpiar()`

Python

```
def dibujar_arbol(self, nodo, x, y, espacio):

    if nodo is None:
        return

    radio = 20

    self.scene.addEllipse(
        x,
        y,
        radio * 2,
        radio * 2,
        QPen(Qt.black),
        QBrush(Qt.white)
    )

    texto = self.scene.addText(str(nodo.value))
    texto.setPos(x + 12, y + 7)
```

Perfecto, entonces ya quedó funcionando el **Árbol de Expresión**. ✅

El error era por el formato de la expresión que esperaba el método (`posfija` con espacios), no por la construcción del árbol.

Hasta ahora tu módulo ya tiene:

- ✅ Carga de la ventana desde el menú principal.
- ✅ Conexión del `QAction` corregida (`actionArbol_expresion`).
- ✅ Construcción del árbol usando tu clase `Stack`.
- ✅ Validación de la expresión.
- ✅ Creación de nodos operadores y operandos.

Antes de darlo por terminado te recomiendo probar estos casos:

1. Expresión sencilla

3 4 +



Resultado esperado:

+
/
3 4



2. Expresión con prioridad

+ Preguntar lo que quieras



ÁRBOL BINARIO DE EXPRESIONES

Expresión Postfija

3 4 +

Construir Árbol

Limpiar

Cerrar

Inorden

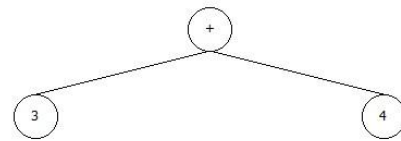
(3 + 4)

Preorden

+ 3 4

Postorden

3 4 +



ÁRBOL BINARIO DE EXPRESIONES

Expresión Postfija

8 2 / 3 4 + *

Construir Árbol

Limpiar

Cerrar

Inorden

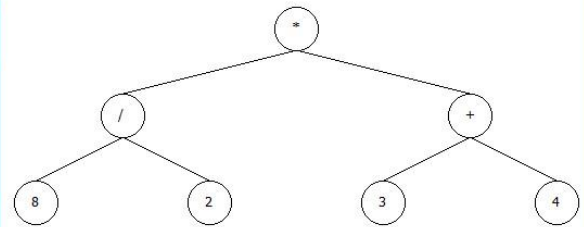
 $((8 / 2) * (3 + 4))$

Preorden

* / 8 2 + 3 4

Postorden

8 2 / 3 4 + *



ÁRBOL BINARIO DE EXPRESIONES

Expresión Postfija

3 4 + 5 2 - *

Construir Árbol

Limpiar

Cerrar

Inorden

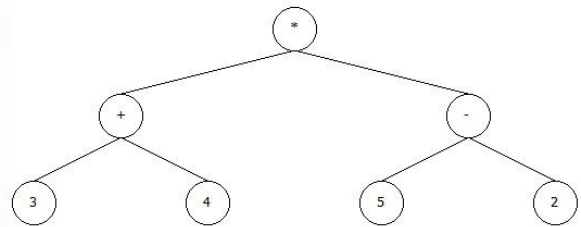
 $((3 + 4) * (5 - 2))$

Preorden

 $* + 3 4 - 5 2$

Postorden

3 4 + 5 2 - *



ÁRBOL BINARIO DE EXPRESIONES

Expresión Postfija

10 2 3 * +

Construir Árbol

Limpiar

Cerrar

Inorden

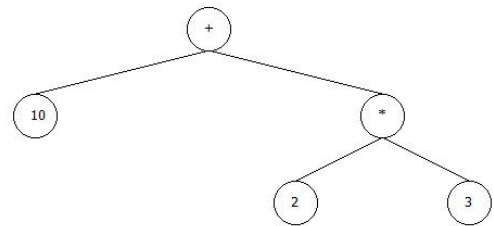
(10 + (2 * 3))

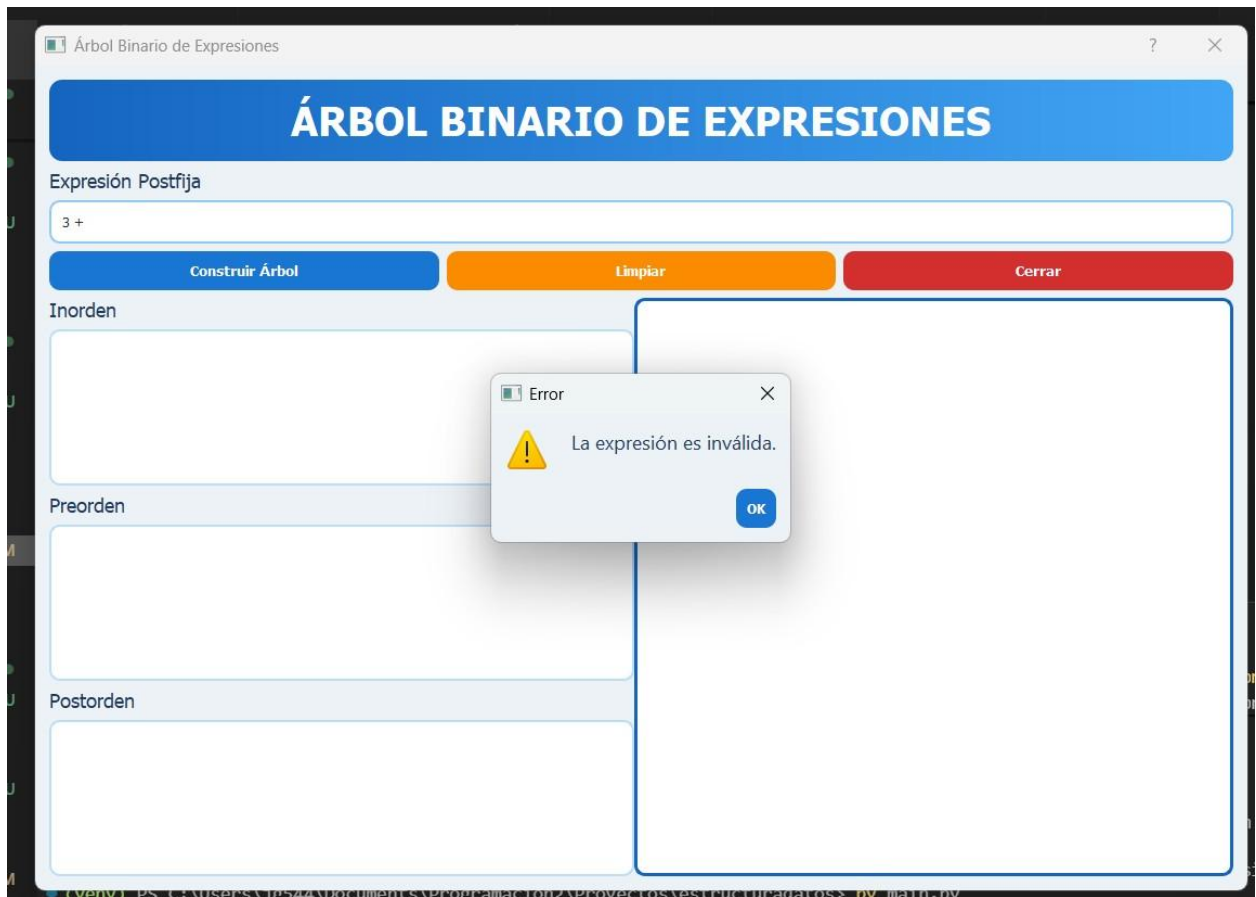
Preorden

+ 10 * 2 3

Postorden

10 2 3 * +





Con la implementación del árbol de expresiones pude entender cómo se puede representar una operación matemática utilizando una estructura de datos diferente a las listas o pilas. Aprendí que los operandos se guardan como nodos y que los operadores sirven para unir los diferentes elementos del árbol. También comprendí la importancia de usar una pila para construir el árbol a partir de una expresión posfija. Este ejercicio me ayudó a reforzar el uso de nodos, clases y recorridos de árboles. En conclusión, los árboles de expresiones permiten organizar y resolver operaciones de una manera más ordenada y facilitan el análisis de expresiones matemáticas